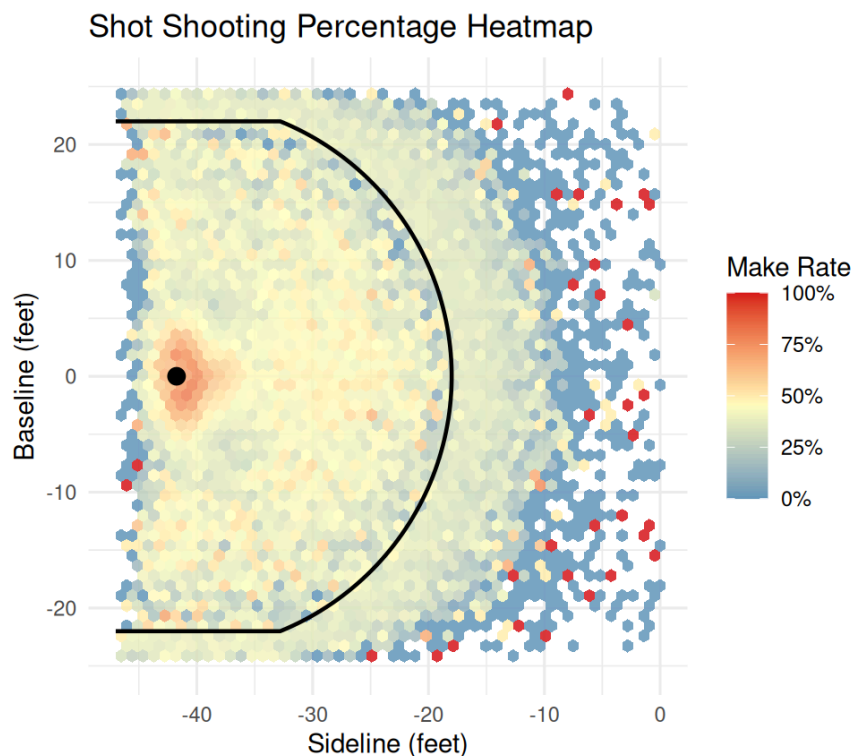


NAME: Derec Gregory #: 2292

This project is something that was surprisingly new for me. I haven't taken a machine learning class yet (it's only offered in the spring semester), so this was a very difficult yet rewarding experience. I have done lots of data analysis when it comes to NBA and shot accuracy, but none when it comes to predictive model building.

In my exploratory data analysis phase, I graphed all of the training dataset into a basketball court graphic so I could have a better picture of what challenges I may face, and what an accurate prediction should look like. This was the result. This will also be used later on in my analysis of accuracy of my model.



I moved forward with an idea: Run multiple logistic regressions to find out which variables were most effective in accurately predicting the outcome of a shot. I started with uni-variate analysis, and found that distance and shot type were the best indicators of what the result of the shot would be. After performing multi-variate analysis on the same variables, I found similar results, but also found that a player's speed went from a negative variable to a positive variable.

After this, I decided to begin data manipulation and created a few new variables. Shooter_efficiency was built to reflect the percentage of shots that a shooter makes. Similarly, defensive_efficiency was designed to display what percentage of shots in which they were the

closest contestant would result in a make. I expected these variables to have a high impact, as players with low shooting percentages would likely miss more shots and vice versa. To my surprise, these variables didn't actually have that great of an impact on any of my models. I believe this is due to two main reasons: the skill floor in the NBA is so high that everyone is very good at shooting, and players who are worse at shooting take far fewer shots. Nevertheless, they had a slight positive impact on the log_loss of each of my models, so they made the cut.

Other variables I created were shot_angle to track the angle to the rim that the shot was taken at, and two trackers to flag if the shot was taken from behind the backboard, and how far behind the backboard it was taken. These also had positive effects on my log_loss.

The other variables I used in my model were dribblesbefore, shotclock, shooterspeed, three, shottype, gamestate, distance, number of contesters, contested, closestdefdist, distcont_1, and def_closing_speed (a variable tracking the speed at which the defender is approaching the shooter). I chose these variables, because after using a recursive variable elimination function that compares the log_loss of my model with each of the variables missing, then recursively dropping the variable that caused the greatest decrease in log_loss, these were the variables that created a positive impact.

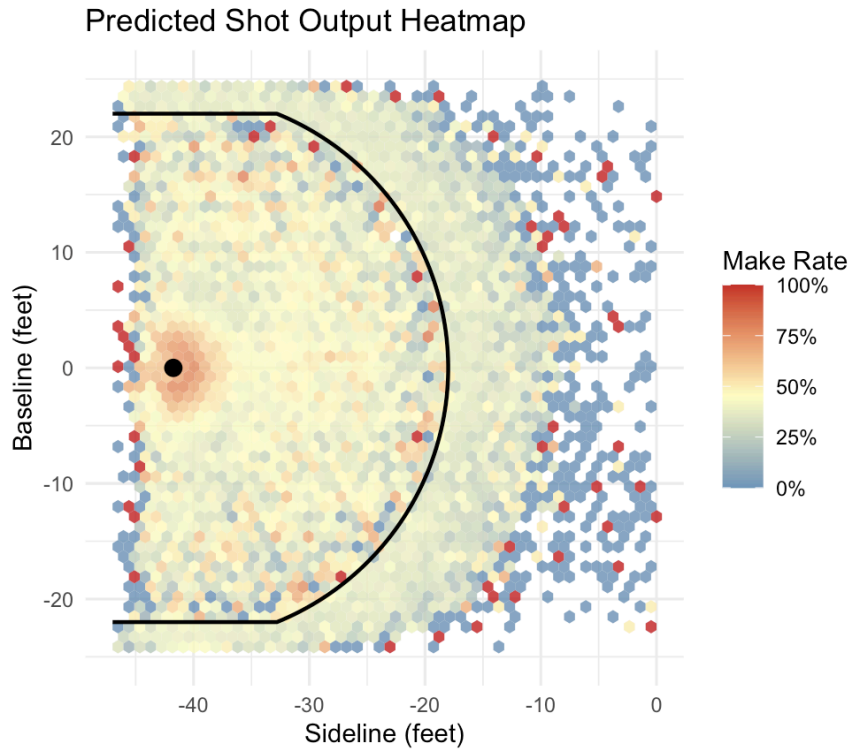
The initial model I used was through Light GBM, an open-source gradient-boosting machine learning framework. After splitting the training dataset into a further split training and testing set, I used this to check the log_loss of my model, and performed the recursive variable elimination function from above. This gave me a log_loss score of about 6.297.

After looking at my model holistically and brainstorming how I could improve it, two main ideas came to my mind. Use a random forest model, and weigh the shots based on distance.

For the latter, I decided to weigh the shots, as since most shots are taken at the rim, this would counteract the model's tendency to care more about shots taken at the rim, and weigh all shots more equally.

Using a random forest model with the same parameters at 100 trees, my model returned a log_loss score of about 6.44, a strong downgrade in quality. So, I decided to combine the predictions of the random forest model and the LightGBM model to see if that would beat the score of the standalone LightGBM model, and it decreased my log_loss score by about .0037, a slight improvement.

Lastly, I ran one simulation of each shot in the testing dataset and predicted if it would be a make or a miss, and got the following graph:



It looks generally similar to the graph of the training dataset that contains the real output of the shots. The things that stuck out to me from this are as follows:

- Shots behind the backboard are predicted to go in at a higher rate than is realistic
 - This could also just be the result of low sample size in that area and a few low-chance shots being predicted to go in.
- A lot of shots just inside the three-point line are projected to go in, more than with the training model.
 - Again, this is likely due to low sample size in this area.

Some other things I built that didn't get included in the final markdown:

- A monte-carlo function to get the mean accuracy of each of my models
- A projected shot accuracy map for both the training dataset and the testing dataset

Overall I'm very happy with my model. It has a good log_loss score, with an improved accuracy from the naive model. If I had more time and data to work with, I would invest more time into fixing behind-the-backboard shots, divide up the shooter_efficiency and defender_efficiency into efficiency by region, and include the team_id as a variable in order to give my model a better sense of individual defense versus team defense.